

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

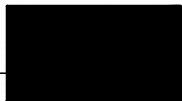
“Информационная безопасность баз данных”

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Реализация сервиса для взаимодействия с разработанной базой данных

Выполнила:

Жук Анастасия Валерьевна,
студент группы N3348


(подпись)

Проверил:

Ярцева Наталия Андреевна,
ассистент ФБИТ

(подпись)

Санкт-Петербург
2025г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	4
Технологический стек.....	4
Архитектура приложения.....	6
Реализация многопользовательского доступа.....	7
Реализация сложных SQL-запросов с GROUP BY.....	19
Запуск приложение и использование интерфейса.....	29
ЗАКЛЮЧЕНИЕ.....	35

ВВЕДЕНИЕ

Цель работы: получение навыков использования серверных языков программирования, фреймворков по работе с БД, ORM-систем.

Для достижения поставленной цели необходимо выполнить следующие **задачи**:

- выбрать и обосновать технологический стек;
- реализовать многопользовательский доступ с ролевой моделью;
- организовать доступ к данным в зависимости от роли;
- реализовать сложные SQL-запросы с GROUP BY;
- создать веб-интерфейс с возможностями CRUD;
- обеспечить информационную безопасность на уровне приложения;
- протестировать получившийся продукт.

ХОД РАБОТЫ

Технологический стек

Для разработки приложения были выбраны следующие технические средства:

- **Python** – современный, безопасный язык с богатой экосистемой для веб-разработки;
- PyCharm – среда программирования;
- **FastAPI** – высокопроизводительный фреймворк с автоматической документацией, поддержкой асинхронности и встроенной валидацией;
- **SQLAlchemy** – мощный ORM для работы с PostgreSQL, поддерживающий сложные запросы и транзакции;
- **JWT + HTTPBearer** для безопасной аутентификации в REST API;
- Дополнительные инструменты:
 - **python-jose** для работы с JWT-токенами;
 - **passlib** и **bcrypt** для хэширования паролей;
 - **Jinja2** для шаблонизации HTML;
 - **uvicorn** – ASGI-сервер для запуска приложения.

Установка всех необходимых библиотек и фреймворков осуществлялась с помощью текстового файла requirements.txt, содержимое которого следующее:

```
fastapi==0.104.1 – веб-фреймворк для создания API на Python
uvicorn==0.24.0 – ASGI-сервер для запуска FastAPI приложений
sqlalchemy==2.0.23 – библиотека для работы с базами данных через
объекты Python
psycopg2-binary==2.9.9 – нативный драйвер для подключения к
PostgreSQL
python-jose[cryptography]==3.3.0 – библиотека для создания и
проверки JWT (JSON Web Tokens)
passlib[bcrypt]==1.7.4 – библиотека для безопасного хэширования
паролей
python-multipart==0.0.6 – библиотека для обработки
multipart/form-data
jinja2==3.1.2 – шаблонизатор для генерации HTML
```

alembic==1.12.1 – инструмент для управления изменениями схемы БД (миграции БД)

pydantic==2.5.0 – библиотека для проверки и преобразования данных

Запустив данный файл с помощью команды *pip install -r requirements.txt* после предварительного перехода в папку с проектом, установятся все необходимые для работы библиотеки и фреймворки.

Также можно установить все зависимости отдельно:

pip install fastapi==0.104.1

pip install uvicorn==0.24.0

pip install sqlalchemy==2.0.23

pip install psycopg2-binary==2.9.9

pip install python-jose[cryptography]==3.3.0

pip install passlib[bcrypt]==1.7.4

pip install python-multipart==0.0.6

pip install jinja2==3.1.2

pip install pydantic==2.5.0

pip install alembic==1.12.1

Архитектура приложения

Все составные части приложения хранятся в папке *access-control-app/*. В этой папке хранятся следующие файлы и папки:

main.py	# Основное приложение FastAPI
auth.py	# Модуль аутентификации и авторизации
models.py	# SQLAlchemy модели (сущности БД)
database.py	# Подключение к БД, сессии
schemas.py	# Pydantic схемы для валидации
config.py	# Конфигурация (маппинг ролей, настройки)
static/	# Статические файлы (CSS, JS)
templates/	# HTML-шаблоны для ролевых дашбордов

Полный код всех составных частей приложения можно будет посмотреть в прикрепленном zip-файле. В каждом файле есть комментарии.

Реализация многопользовательского доступа

В *config.py* и *auth.py* реализован гибкий маппинг должностей из таблицы *roles* на роли доступа в приложении:

```
# Маппинг должностей на роли доступа в приложении
# Ключ: название должности в таблице roles.role_name
# Значение: роль доступа в приложении (ib_admin,
team_lead, employee, auditor, sec_service)
ROLE_MAPPING = {
    # Административные роли
    "Администратор ИБ": "ib_admin",
    "Служба безопасности": "sec_service",
    "Аудитор": "auditor",

    # Руководящие роли
    "Тимлид": "team_lead",
    "Руководитель отдела": "team_lead",
    "Руководитель проекта": "team_lead",

    # Технические роли (сотрудники)
    "DevOps-инженер": "employee",
    "Разработчик": "employee",
    "Тестировщик": "employee",
    "Аналитик": "employee",
    "Дизайнер": "employee",

    # Начальные позиции
    "Стажер": "employee",
    "Младший разработчик": "employee",

    # По умолчанию для всех остальных должностей
}

# Роль по умолчанию, если должность не найдена в маппинге
DEFAULT_ROLE = "employee"
```

```
# ===== МАППИНГ РОЛЕЙ =====
ROLE_MAPPING = {
    "Администратор ИБ": "ib_admin",
    "Служба безопасности": "sec_service",
    "Аудитор": "auditor",
    "Тимлид": "team_lead",
```

```

    "Руководитель отдела": "team_lead",
    "DevOps-инженер": "employee",
    "Разработчик": "employee",
    "Стажер": "employee",
}

```

```

DEFAULT_ROLE = "employee"

```

В процессе появления новых ролей сотрудников, администратор информационной безопасности сможет дополнять эту часть кода приложения.

Аутентификация происходит через JWS-токен, чтобы не запрашивать постоянно у сотрудника его логин и пароль. Пароли хранятся в БД в захешированном виде с использованием *crypt()* PostgreSQL. При успешном входе генерируется JWT-токен с полями *sub* (логин), *role*, *exp* (срок жизни токена составляет 30 минут, после чего система просит пользователя заново авторизоваться). Токен сохраняется в таблицу *auth_tokens* и передается клиенту через *cookie*.

```

@app.post("/login")
async def login_user(
    request: Request,
    username: str = Form(...),
    password: str = Form(...),
    db: Session = Depends(get_db)
):
    """Обработка входа пользователя"""
    # Аутентифицируем пользователя
    employee = auth.authenticate_user(db, username,
password)
    if not employee:
        return templates.TemplateResponse("login.html", {
            "request": request,
            "error": "Неверное имя пользователя или
пароль"
        })

    # Создаем и сохраняем токен
    token_data = await auth.create_and_store_token(db,
employee)
    access_token = token_data["access_token"]
    access_role = token_data["role"]

```

```

        # Создаем HTML с JavaScript для установки cookie и
        редиректа
        html_content = f"""
        <!DOCTYPE html>
        <html>
        <head>
            <title>Успешный вход</title>
            <script>
                // Устанавливаем токен в cookie
                document.cookie =
"access_token={access_token}; path=/; max-age=1800;
SameSite=Lax";

                // Через небольшой таймаут делаем редирект
(чтобы cookie успел установиться)
                setTimeout(function() {{
                    window.location.href =
"/dashboard/{access_role}?token={access_token}";
                }}, 100);
            </script>
        </head>
        <body>
            <div style="text-align: center; margin-top:
100px;">

                <h2>Вход выполнен успешно!</h2>
                <p>Перенаправление на панель управления...</p>
                <p><small>Если перенаправление не произошло,
<a
href="/dashboard/{access_role}?token={access_token}">нажмите
сюда</a></small></p>
            </div>
        </body>
        </html>
        """

        return HTMLResponse(content=html_content)
@app.get("/api/check_token")
async def check_token(
    request: Request,
    db: Session = Depends(get_db)
):
    """Проверяет валидность токена"""
    # Пробуем получить токен из разных источников
    token = None

```

```

# 1. Из Authorization header
auth_header = request.headers.get("Authorization")
if auth_header and auth_header.startswith("Bearer "):
    token = auth_header.split(" ")[1]

# 2. Из cookie
if not token:
    token = request.cookies.get("access_token")

# 3. Из query параметра
if not token:
    token = request.query_params.get("token")

if not token:
    return JsonResponse(
        status_code=200, # Используем 200 чтобы
        # фронтенд мог обработать
        content={
            "valid": False,
            "message": "Токен не предоставлен",
            "redirect_url": "/"
        }
    )

# Проверяем токен
try:
    employee = await auth.validate_token(db, token)

    if employee:
        return {
            "valid": True,
            "message": "Токен действителен",
            "user": {
                "login": employee.login,
                "full_name": employee.full_name,
                "role":
auth.get_user_access_role(employee)
            },
            "timestamp": datetime.now().isoformat()
        }
    else:
        return JsonResponse(
            status_code=200,

```

```

        content={
            "valid": False,
            "message": "Токен недействителен или истек",
            "redirect_url": "/"
        }
    )

except Exception as e:
    print(f"Ошибка проверки токена: {e}")
    return JsonResponse(
        status_code=200,
        content={
            "valid": False,
            "message": "Ошибка проверки токена",
            "redirect_url": "/"
        }
    )

```

```

def verify_password_postgres(db: Session, plain_password: str, hashed_password: str) -> bool:
    """Проверяет пароль через PostgreSQL crypt()"""
    try:
        query = text("SELECT crypt(:password, :hash) = :hash as is_valid")
        result = db.execute(
            query,
            {"password": plain_password, "hash": hashed_password}
        ).fetchone()
        return result.is_valid if result else False
    except Exception as e:
        print(f"Ошибка проверки пароля: {e}")
        return False

def authenticate_user(db: Session, username: str, password: str) -> Optional[models.Employee]:
    """Аутентифицирует пользователя"""
    employee = db.query(models.Employee).filter(models.Employee.login == username).first()

```

```

        if not employee:
            print(f"Пользователь {username} не найден")
            return None

        if verify_password_postgres(db, password,
employee.password):
            return employee

        print(f"Неверный пароль для {username}")
        return None

def get_user_access_role(employee: models.Employee) ->
str:
    """Определяет роль доступа на основе должности"""
    if not employee or not employee.role:
        return DEFAULT_ROLE

    role_name = employee.role.role_name
    return ROLE_MAPPING.get(role_name, DEFAULT_ROLE)

def create_jwt_token(login: str, role: str) -> str:
    """Создает JWT токен"""
    expires_delta =
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    expire = datetime.now() + expires_delta

    to_encode = {
        "sub": login,
        "role": role,
        "exp": int(expire.timestamp())
    }

    return jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)

async def create_and_store_token(db: Session, employee:
models.Employee) -> Dict[str, str]:
    """Создает JWT токен и сохраняет его в БД, удаляя
старые токены пользователя"""
    try:
        # Определяем роль

```

```

        access_role = get_user_access_role(employee)

        # УДАЛЯЕМ ВСЕ СУЩЕСТВУЮЩИЕ ТОКЕНЫ ПОЛЬЗОВАТЕЛЯ
        deleted_count = db.query(models.AuthToken).filter(
            models.AuthToken.user_login == employee.login
        ).delete()

        if deleted_count > 0:
            print(f"Удалено {deleted_count} старых токенов
для пользователя {employee.login}")

        # Создаем новый JWT токен
        jwt_token = create_jwt_token(employee.login,
access_role)

        # Создаем запись токена в БД
        expires_at = datetime.now() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

        auth_token = models.AuthToken(
            user_login=employee.login,
            encrypted_token=jwt_token,
            expires_at=expires_at
        )

        db.add(auth_token)
        db.commit()

        return {
            "access_token": jwt_token,
            "token_type": "bearer",
            "role": access_role,
            "employee_id": employee.id_employee
        }

    except Exception as e:
        db.rollback()
        print(f"Ошибка создания токена: {e}")
        raise

    async def validate_token(db: Session, token: str) ->
Optional[models.Employee]:
        """Проверяет токен и возвращает сотрудника"""

```

```

try:
    # 1. Проверяем в БД
    auth_token = db.query(models.AuthToken).filter(
        models.AuthToken.encrypted_token == token,
        models.AuthToken.expires_at > datetime.now()
    ).first()

    if not auth_token:
        print("Токен не найден в БД или истек")
        return None

    # 2. Проверяем JWT
    payload = jwt.decode(token, SECRET_KEY,
algorithm=[ALGORITHM])
    user_login = payload.get("sub")

    if user_login != auth_token.user_login:
        print("Несоответствие ID сотрудника в токене")
        return None

    # 3. Получаем сотрудника
    employee = db.query(models.Employee).filter(
        models.Employee.login == user_login
    ).first()

    return employee

except JWTError as e:
    print(f"Ошибка JWT: {e}")
    return None
except Exception as e:
    print(f"Ошибка проверки токена: {e}")
    return None

```

FastAPI позволяет создавать зависимости для проверки ролей:

```

def require_role(required_role: str):
    """Создает dependency для проверки роли"""

    async def role_checker(
        current_user: models.Employee =
Depends(get_current_user),
        db: Session = Depends(get_db)
    ) -> models.Employee:
        user_role = get_user_access_role(current_user)

```

```

        if user_role != required_role:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Недостаточно прав. Требуется
роль: {required_role}, а у вас: {user_role}"
            )
        return current_user

    return role_checker

# Конкретные зависимости для каждой роли
check_ib_admin = require_role("ib_admin")
check_team_lead = require_role("team_lead")
check_employee = require_role("employee")
check_auditor = require_role("auditor")
check_sec_service = require_role("sec_service")

```

Каждая роль получает уникальный интерфейс (/dashboard/{role_type}):

- Администратор ИБ – полный доступ ко всем таблицам, логам, ресурсам;
- Руководитель команды – просмотр состава команды, доступов команды;
- Сотрудник – личная информация, история доступов;
- Аудитор – подозрительные доступы, аудит-логи;
- Служба безопасности – потенциальные прогулы, нарушения.

```

# ===== ПАНЕЛИ УПРАВЛЕНИЯ =====

@app.get("/dashboard/{role_type}",
response_class=HTMLResponse)
async def dashboard(
    request: Request,
    role_type: str,
    db: Session = Depends(get_db),
    current_user: models.Employee =
Depends(auth.get_current_user)
):
    """Панель управления в зависимости от роли"""
    # Проверяем, что роль пользователя совпадает с
запрошенной
    user_role = auth.get_user_access_role(current_user)

```

```

        if user_role != role_type:
            raise HTTPException(
                status_code=403,
                detail=f"Доступ запрещен. Ваша роль: {user_role}, запрошена: {role_type}"
            )

    # Подготавливаем контекст
    context = {
        "request": request,
        "user": current_user,
        "user_role": user_role,
        "role_type": role_type
    }

    # Заполняем данными в зависимости от роли
    if role_type == "ib_admin":
        # Администратор ИБ видит все
        context["employees"] = db.query(models.Employee).limit(100).all()
        context["roles"] = db.query(models.Role).all()
        context["teams"] = db.query(models.Team).all()
        context["statuses"] = db.query(models.EmployeeStatus).limit(50).all()
        context["access_logs"] = db.query(models.AccessLog).order_by(models.AccessLog.event_date.desc()).limit(100).all()
        context["physical_resources"] = db.query(models.PhysicalResource).all()
        context["information_resources"] = db.query(models.InformationResource).all()
        context["audit_logs"] = db.query(models.AccessAuditLog).order_by(models.AccessAuditLog.operation_timestamp.desc()).limit(50).all()

    elif role_type == "team_lead":
        # Руководитель команды видит свою команду
        if current_user.team:
            team_members = db.query(models.Employee).filter(
                models.Employee.id_team ==

```

```

current_user.id_team
    ).all()
    context["team_members"] = team_members
    context["team"] = current_user.team

    # Доступы команды к ресурсам
    team_access =
db.query(models.TeamResourceAccess).filter(
    models.TeamResourceAccess.id_team ==
current_user.id_team
    ).all()
    context["team_access"] = team_access

    elif role_type == "employee":
        # Сотрудник видит только свою информацию
        context["employee"] = current_user
        if current_user.status:
            context["status"] = current_user.status

        # История доступов сотрудника
        access_history =
db.query(models.AccessLog).filter(
    models.AccessLog.id_employee ==
current_user.id_employee

).order_by(models.AccessLog.event_date.desc()).limit(50).all()
    context["access_history"] = access_history

    elif role_type == "auditor":
        # Аудитор видит подозрительные доступы
        suspicious_access =
db.query(models.AccessLog).filter(
    models.AccessLog.result == "отказано"

).order_by(models.AccessLog.event_date.desc()).limit(100).all(
)
    context["suspicious_access"] = suspicious_access

    # Аудит-логи
    audit_logs =
db.query(models.AccessAuditLog).order_by(
models.AccessAuditLog.operation_timestamp.desc()
    ).limit(100).all()

```

```

        context["audit_logs"] = audit_logs

    elif role_type == "sec_service":
        # Служба безопасности видит потенциальные проблемы
        # Потенциальные прогулы
        potential_absenteeism =
db.query(models.EmployeeStatus).filter(
    and_(
        models.EmployeeStatus.status_type ==
"Активная работа",
        models.EmployeeStatus.employee_active == 0
    )
).all()
        context["potential_absenteeism"] =
potential_absenteeism

        # Последние нарушения
        recent_violations =
db.query(models.AccessLog).filter(
    models.AccessLog.result == "отказано"
).order_by(models.AccessLog.event_date.desc()).limit(50).all()
        context["recent_violations"] = recent_violations

    return
templates.TemplateResponse(f"dashboard_{role_type}.html",
context)

```

Реализация сложных SQL-запросов с GROUP BY

Для каждой роли реализованы статистические отчеты с возможностью сортировки и фильтрации.

Для администратора ИБ: статистика по ролям (роли — сотрудники — команды).

Для аудитора: статистика доступа по статусам (статусы — сотрудники — журнал доступа).

Для службы безопасности: статистика нарушений (журнал доступа — типы нарушений — анализ серьезности).

Для руководителя команды: статистика команды (сотрудники команды — роли — журнал доступа).

Для сотрудника: персональная статистика (сотрудник — журнал доступа — анализ успешности).

```
# ===== СТАТИСТИКА =====

@app.get("/stats/{role_type}",
response_class=HTMLResponse)
async def stats_page(
    request: Request,
    role_type: str,
    sort_by: Optional[str] = Query(None),
    order: Optional[str] = Query("desc"),
    start_date: Optional[str] = Query(None),
    end_date: Optional[str] = Query(None),
    db: Session = Depends(get_db),
    current_user: models.Employee =
Depends(auth.get_current_user)
):
    """Универсальная страница статистики для всех ролей"""
    user_role = auth.get_user_access_role(current_user)

    # Проверяем доступ
    if user_role != role_type:
        raise HTTPException(status_code=403,
detail="Доступ запрещен")

    context = {
```

```

        "request": request,
        "user": current_user,
        "user_role": user_role,
        "role_type": role_type,
        "zip": zip
    }

    # ОБРАБОТКА ВСЕХ РОЛЕЙ В ОДНОМ МЕСТЕ
    if role_type == "employee":
        # Код из employee_stats_page
        total_access = db.query(models.AccessLog).filter(
            models.AccessLog.id_employee ==
current_user.id_employee
        ).count()

        successful_access =
db.query(models.AccessLog).filter(
            and_(
                models.AccessLog.id_employee ==
current_user.id_employee,
                models.AccessLog.result == "успешно"
            )
        ).count()

        failed_access = db.query(models.AccessLog).filter(
            and_(
                models.AccessLog.id_employee ==
current_user.id_employee,
                models.AccessLog.result == "отказано"
            )
        ).count()

        success_rate = round((successful_access /
total_access * 100) if total_access > 0 else 0, 2)

        recent_activities_query =
db.query(models.AccessLog).filter(
            models.AccessLog.id_employee ==
current_user.id_employee
        ).order_by(models.AccessLog.event_date.desc()).limit(10).all()

        recent_activities = []
        for activity in recent_activities_query:

```

```

        recent_activities.append({
            "date":
activity.event_date.strftime("%Y-%m-%d %H:%M"),
            "title": f"{activity.action} -
{activity.result}",
            "description": activity.reason or "Без
описания"
        })

        # Используем шаблон stats_employee.html для
employee
context.update({
    "stats": {
        "total_access": total_access,
        "successful": successful_access,
        "failed": failed_access,
        "success_rate": success_rate
    },
    "recent_activities": recent_activities
})

return
templates.TemplateResponse("stats_employee.html", context)

elif role_type == "ib_admin":
    stats = await get_role_stats(sort_by or
"employee_count", order or "desc", db, current_user)
    context["stats_data"] = stats
    context["stats_title"] = "Статистика по ролям"
    context["columns"] = ["role_name", "access_role",
"employee_count", "avg_security_level",
                                "in_team_count",
"in_team_percentage"]
    context["column_names"] = ["Должность", "Роль
доступа", "Кол-во сотрудников",
                                "Ср. уровень
безопасности", "В командах", "% в командах"]

    return templates.TemplateResponse("stats.html",
context)

elif role_type == "auditor":
    # Исправляем join в запросе
    query = db.query(

```

```

        models.EmployeeStatus.status_type,

func.count(models.AccessLog.event_id).label("total_access"),
        func.count(case((models.AccessLog.result ==
"успешно", 1))).label("successful"),
        func.count(case((models.AccessLog.result ==
"отказано", 1))).label("failed")
        ).join(models.Employee,
models.Employee.id_employee == models.AccessLog.id_employee) \
        .join(models.EmployeeStatus,
models.EmployeeStatus.id_employee ==
models.Employee.id_employee)

        if start_date:
            try:
                start_date_obj =
datetime.strptime(start_date, "%Y-%m-%d")
                query =
query.filter(models.AccessLog.event_date >= start_date_obj)
            except ValueError:
                pass

            if end_date:
                try:
                    end_date_obj = datetime.strptime(end_date,
"%Y-%m-%d")
                    query =
query.filter(models.AccessLog.event_date <= end_date_obj)
                except ValueError:
                    pass

            query =
query.group_by(models.EmployeeStatus.status_type)

        # Сортировка
        if sort_by == "total_access":
            query = query.order_by(desc("total_access") if
order == "desc" else asc("total_access"))
        elif sort_by == "successful":
            query = query.order_by(desc("successful") if
order == "desc" else asc("successful"))
        elif sort_by == "failed":
            query = query.order_by(desc("failed") if order
== "desc" else asc("failed"))

```

```

        elif sort_by == "status_type":
            query = query.order_by(
                desc(models.EmployeeStatus.status_type) if
order == "desc" else asc(models.EmployeeStatus.status_type)
            )

        stats = query.all()

        result = []
        for stat in stats:
            total = int(stat.total_access) if
stat.total_access else 0
            successful = int(stat.successful) if
stat.successful else 0
            failed = int(stat.failed) if stat.failed else
0

            result.append({
                "status_type": stat.status_type,
                "total_access": total,
                "successful": successful,
                "failed": failed,
                "success_rate": round((successful / total
* 100) if total > 0 else 0, 2),
                "failure_rate": round((failed / total *
100) if total > 0 else 0, 2)
            })

        # Сортировка результатов если запрос уже
отсортирован
        if sort_by in ["total_access", "successful",
"failed", "success_rate", "failure_rate"]:
            reverse = order == "desc"
            if sort_by == "total_access":
                result.sort(key=lambda x:
x["total_access"], reverse=reverse)
            elif sort_by == "successful":
                result.sort(key=lambda x: x["successful"],
reverse=reverse)
            elif sort_by == "failed":
                result.sort(key=lambda x: x["failed"],
reverse=reverse)
            elif sort_by == "success_rate":
                result.sort(key=lambda x:

```

```

x["success_rate"], reverse=reverse)
        elif sort_by == "failure_rate":
            result.sort(key=lambda x:
x["failure_rate"], reverse=reverse)

        context["stats_data"] = result
        context["stats_title"] = "Статистика доступа по
статусам"

        context["columns"] = ["status_type",
"total_access", "successful", "failed",
                                "success_rate",
"failure_rate"]
        context["column_names"] = ["Статус", "Всего
попыток", "Успешных", "Отказано",
                                "% успеха", "% отказа"]

        return templates.TemplateResponse("stats.html",
context)

    elif role_type == "team_lead":
        # Получаем статистику команды
        if current_user.id_team and current_user.team:
            # Статистика по членам команды
            team_stats = db.query(
                models.Employee.id_employee,
                models.Employee.full_name,
                models.Role.role_name,

func.count(models.AccessLog.event_id).label("total_access"),
                func.count(case((models.AccessLog.result
== "успешно", 1))).label("successful")
            ).join(models.Role, models.Employee.id_role ==
models.Role.id_role) \
                .join(models.AccessLog,
models.AccessLog.id_employee == models.Employee.id_employee,
isouter=True) \
                .filter(models.Employee.id_team ==
current_user.id_team) \
                .group_by(models.Employee.id_employee,
models.Employee.full_name, models.Role.role_name) \
                .all()

            stats_data = []
            for stat in team_stats:

```

```

        total = int(stat.total_access) if
stat.total_access else 0
        successful = int(stat.successful) if
stat.successful else 0
        failed = total - successful
        success_rate = round((successful / total *
100) if total > 0 else 0, 2)
        failure_rate = round((failed / total *
100) if total > 0 else 0, 2)

        stats_data.append({
            "team_member": stat.full_name,
            "role": stat.role_name,
            "total_access": total,
            "successful": successful,
            "failed": failed,
            "success_rate": success_rate,
            "failure_rate": failure_rate
        })

    # Сортировка
        if sort_by in ["team_member", "role",
"total_access", "successful", "failed", "success_rate",
"failure_rate"]:
            reverse = order == "desc"
            if sort_by == "team_member":
                stats_data.sort(key=lambda x:
x["team_member"], reverse=reverse)
            elif sort_by == "role":
                stats_data.sort(key=lambda x:
x["role"], reverse=reverse)
            elif sort_by == "total_access":
                stats_data.sort(key=lambda x:
x["total_access"], reverse=reverse)
            elif sort_by == "successful":
                stats_data.sort(key=lambda x:
x["successful"], reverse=reverse)
            elif sort_by == "failed":
                stats_data.sort(key=lambda x:
x["failed"], reverse=reverse)
            elif sort_by == "success_rate":
                stats_data.sort(key=lambda x:
x["success_rate"], reverse=reverse)
            elif sort_by == "failure_rate":

```

```

stats_data.sort(key=lambda x:
x["failure_rate"], reverse=reverse)

    context["stats_data"] = stats_data
    context["stats_title"] = f"Статистика команды
{current_user.team.team_name}"
    context["columns"] = ["team_member", "role",
"total_access", "successful", "failed", "success_rate",
                        "failure_rate"]
    context["column_names"] = ["Сотрудник",
"Должность", "Всего доступов", "Успешных", "Отказано", "%
успеха",
                                "% отказа"]

    else:
        context["stats_data"] = []
        context["stats_title"] = "Статистика команды"
        context["columns"] = ["team_member", "role",
"total_access", "success_rate"]
        context["column_names"] = ["Сотрудник",
"Должность", "Всего доступов", "% успеха"]

    return templates.TemplateResponse("stats.html",
context)

elif role_type == "sec_service":
    # Статистика нарушений безопасности
    violation_stats = db.query(
        models.AccessLog.action,

func.count(models.AccessLog.event_id).label("count"),
        # Используем выражение для расчета severity
как float
        (func.avg(
            case((models.AccessLog.result ==
"успешно", 0), else_=1)
            ) * 100).label("severity_percent") # Умножаем
сразу на 100 чтобы получить проценты
        ).filter(models.AccessLog.result == "отказано") \
        .group_by(models.AccessLog.action) \
        .order_by(desc("count")) \
        .all()

    stats_data = []
    for stat in violation_stats:

```

```

        count_val = int(stat.count) if stat.count else
0
        # Конвертируем Decimal во float безопасно
        if stat.severity_percent is not None:
            try:
                severity_val =
float(stat.severity_percent)
            except (TypeError, ValueError):
                severity_val = 0.0
        else:
            severity_val = 0.0

        stats_data.append({
            "violation_type": stat.action,
            "count": count_val,
            "severity": round(severity_val, 2) # Уже
в процентах
        })

        # Сортировка
        if sort_by in ["violation_type", "count",
"severity"]:
            reverse = order == "desc"
            if sort_by == "violation_type":
                stats_data.sort(key=lambda x:
x["violation_type"], reverse=reverse)
            elif sort_by == "count":
                stats_data.sort(key=lambda x: x["count"],
reverse=reverse)
            elif sort_by == "severity":
                stats_data.sort(key=lambda x:
x["severity"], reverse=reverse)

        context["stats_data"] = stats_data
        context["stats_title"] = "Статистика нарушений
безопасности"
        context["columns"] = ["violation_type", "count",
"severity"]
        context["column_names"] = ["Тип нарушения",
"Количество", "Серьезность (%)"]

        return templates.TemplateResponse("stats.html",
context)

```

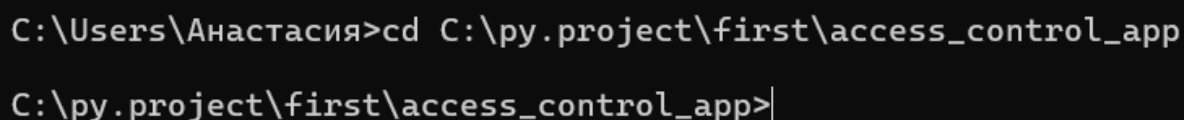
```
    else:
        # Для неизвестных ролей
        raise HTTPException(status_code=404,
detail="Страница статистики не найдена")
```

Запуск приложения и использование интерфейса

Запуск приложения осуществляется следующим образом (запуск будет осуществляться в операционной системе Windows, а демонстрация приложения – в поисковой системе Chrome).

1. Открываем консоль и переходим в папку, в которой лежит проект:

```
cd C:\путь_до_папки_с_приложением\access_control_app
```

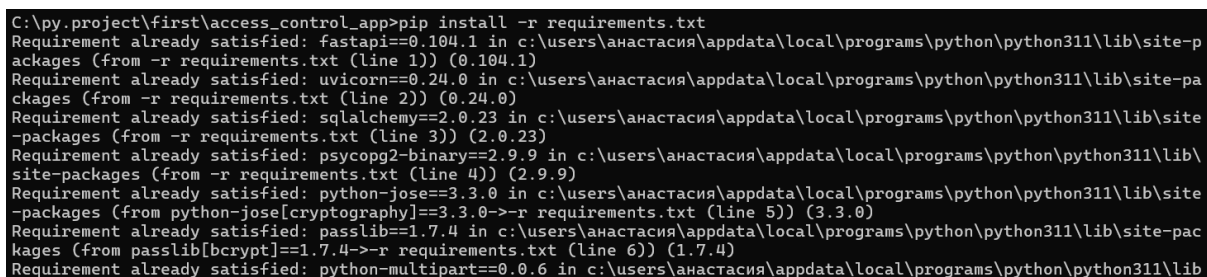


```
C:\Users\Анастасия>cd C:\py.project\first\access_control_app  
C:\py.project\first\access_control_app>|
```

Рисунок 1 – Переход в папку с приложением

2. Устанавливаем зависимости:

```
pip install -r requirements.txt
```

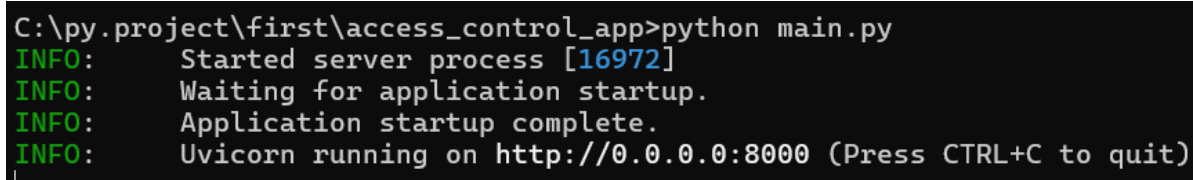


```
C:\py.project\first\access_control_app>pip install -r requirements.txt  
Requirement already satisfied: fastapi==0.104.1 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-packages (from -r requirements.txt (line 1)) (0.104.1)  
Requirement already satisfied: uvicorn==0.24.0 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-packages (from -r requirements.txt (line 2)) (0.24.0)  
Requirement already satisfied: sqlalchemy==2.0.23 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-packages (from -r requirements.txt (line 3)) (2.0.23)  
Requirement already satisfied: pycopp2-binary==2.9.9 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-packages (from -r requirements.txt (line 4)) (2.9.9)  
Requirement already satisfied: python-jose==3.3.0 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-packages (from python-jose[cryptography]==3.3.0->-r requirements.txt (line 5)) (3.3.0)  
Requirement already satisfied: passlib==1.7.4 in c:\users\анастасия\appdata\local\programs\python\python311\lib\site-packages (from passlib[bcrypt]==1.7.4->-r requirements.txt (line 6)) (1.7.4)  
Requirement already satisfied: python-multipart==0.0.6 in c:\users\анастасия\appdata\local\programs\python\python311\lib
```

Рисунок 2 – Установка зависимостей

3. Запускаем приложение:

```
python main.py
```



```
C:\py.project\first\access_control_app>python main.py  
INFO: Started server process [16972]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Рисунок 3 – Приложение запущено

4. Теперь в браузере переходим по адресу localhost:8000 и видим окно авторизации для пользователей:

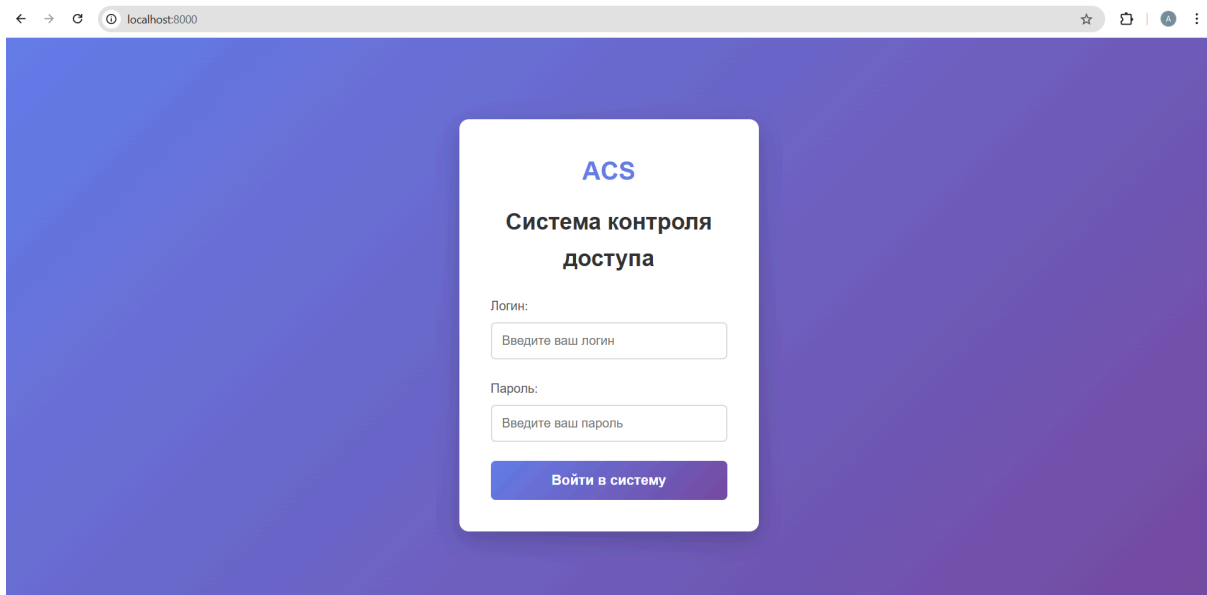


Рисунок 4 – Окно авторизации сотрудников

5. Далее можно войти под любым пользователем из таблицы employees созданной базы данных. Посмотрим, как выглядят интерфейсы для каждой из ролей.

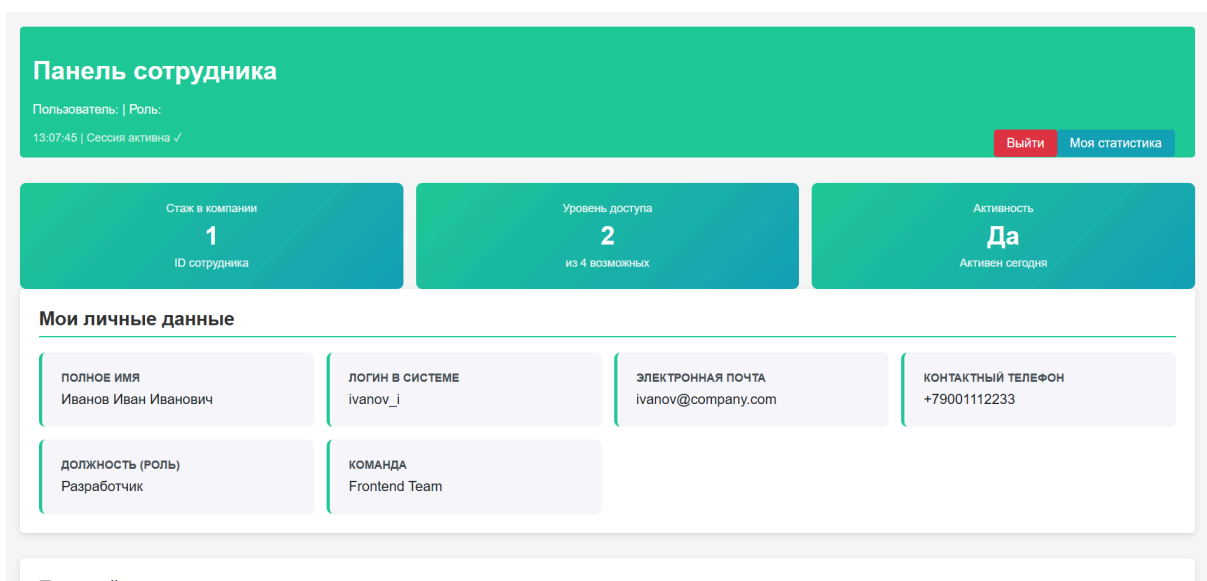


Рисунок 5 – Дашборд сотрудника Иванова Ивана Ивановича



Рисунок 6 – Статистика сотрудника Иванова Ивана Ивановича

Панель администратора информационной безопасности

Пользователь: | Роль:

13:10:57 | Сессия активна ✓

[Выйти](#) [Статистика](#)

Сотрудники

ID	Логин	ФИО	Email	Роль	Команда
1	ivanov_i	Иванов Иван Иванович	ivanov@company.com	Разработчик	Frontend Team
2	petrov_p	Петров Петр Петрович	petrov@company.com	Тимлид	Backend Team
3	sidorov_s	Сидоров Сидор Сидорович	sidorov@company.com	DevOps-инженер	DevSecOps
4	admin_ib	Козлов Геннадий Андреевич	admin_ib@company.com	Администратор ИБ	Нет
5	sec_officer	Васьков Владимир Васильевич	sec@company.com	Служба безопасности	Нет
6	auditor1	Дементьев Сергей Данилович	audit@company.com	Аудитор	Нет
7	manager_a	Кривова Анна Валерьевна	manager@company.com	Руководитель отдела	Frontend Team
8	vacation_user	Прокофьев Денис Иванович	ex@company.com	Разработчик	Нет

Рисунок 7 – Дашборд администратора ИБ Козлова Геннадия Андреевича

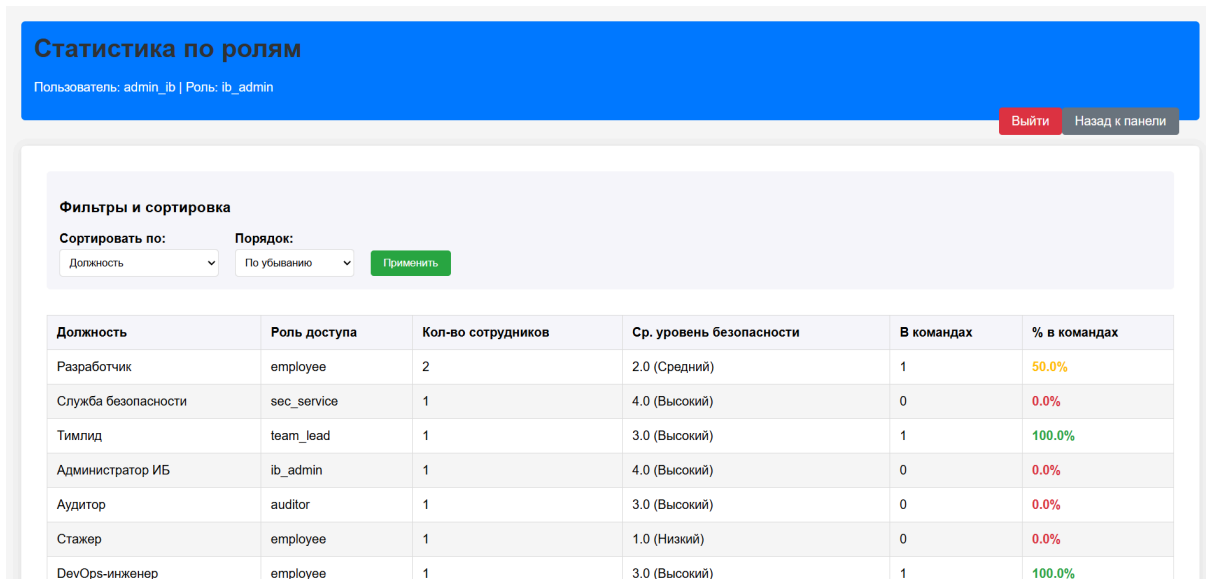


Рисунок 8 – Статистика администратора ИБ Козлова Геннадия Андреевича

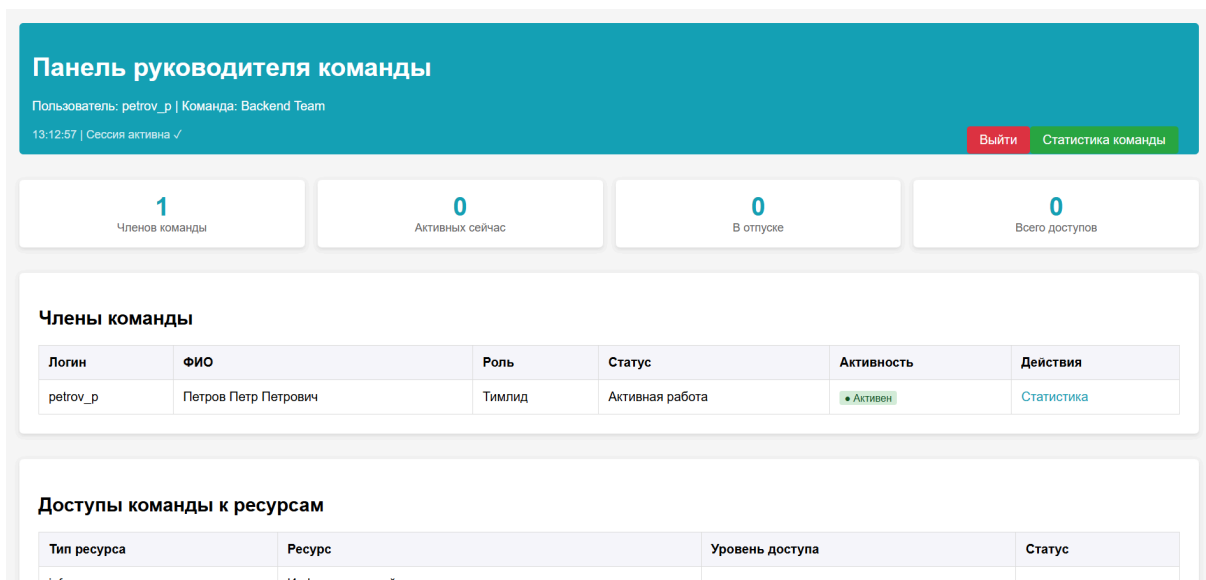


Рисунок 9 – Дашборд руководителя команды Петрова Петра Петровича

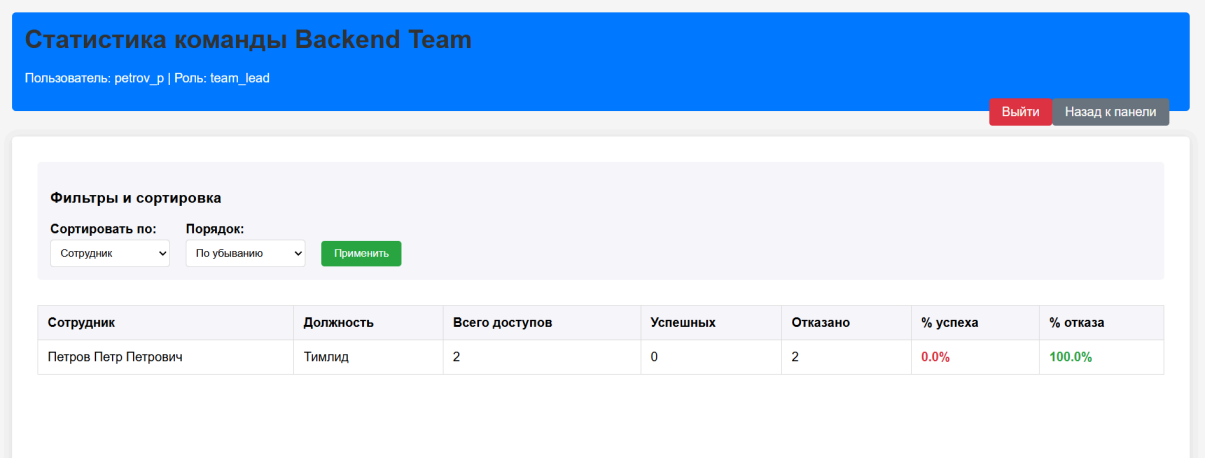


Рисунок 10 – Статистика руководителя команды Петрова Петра Петровича

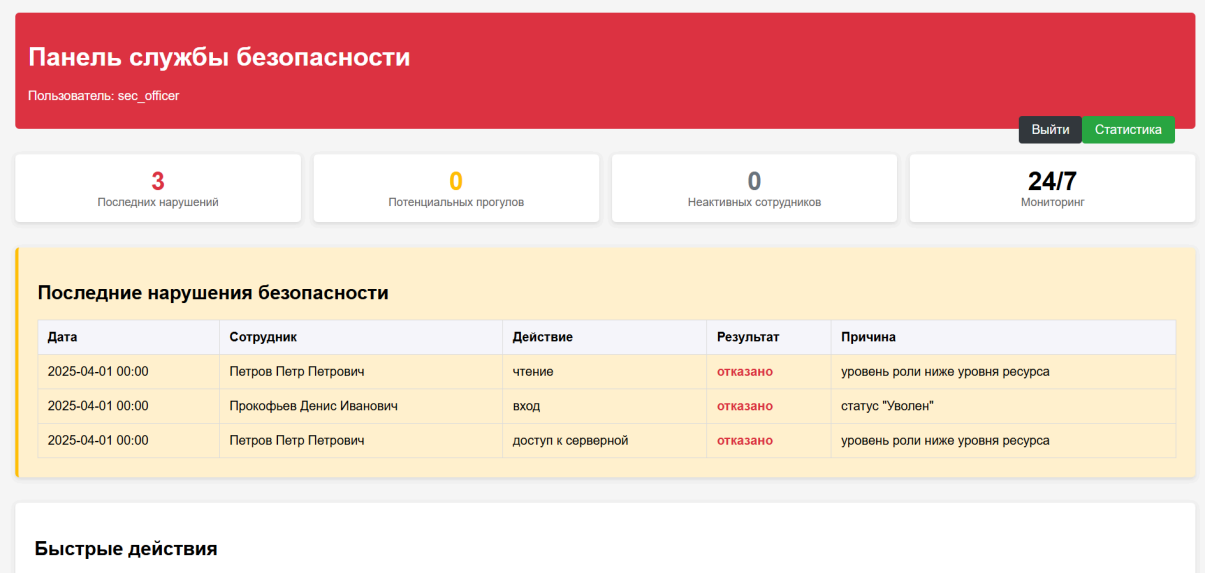


Рисунок 11 – Дашборд представителя службы безопасности Васькова Владимира Васильевича

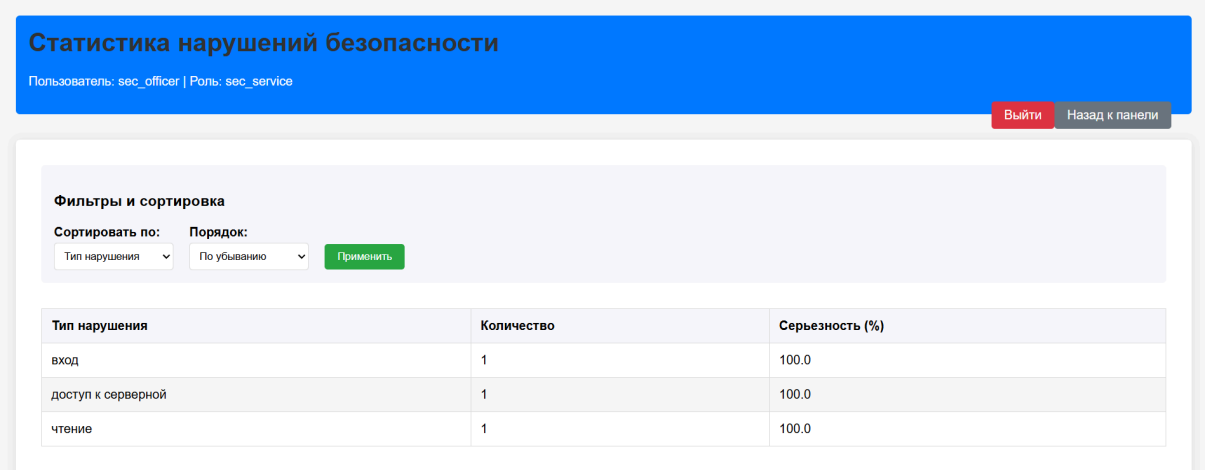


Рисунок 12 – Статистика представителя службы безопасности Васькова Владимира Васильевича

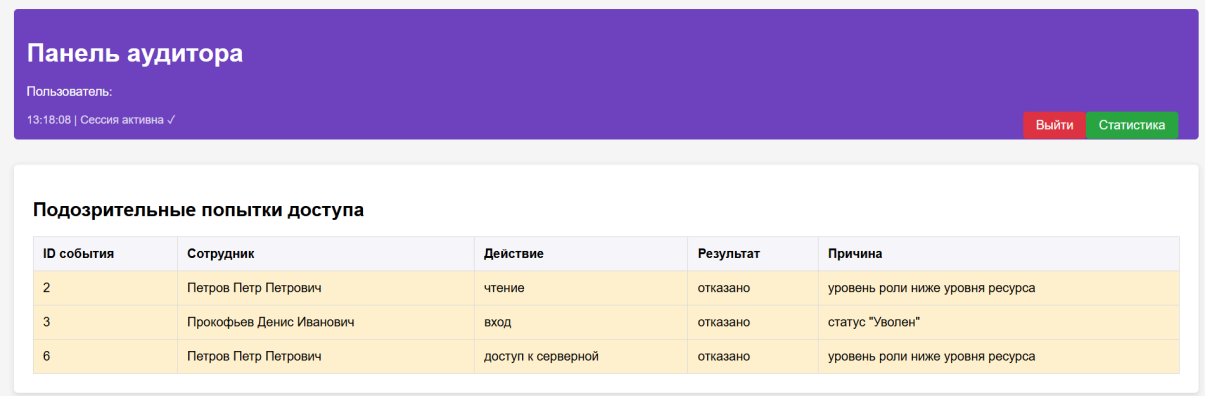


Рисунок 13 – Дашборд аудитора Дементьева Сергея Даниловича

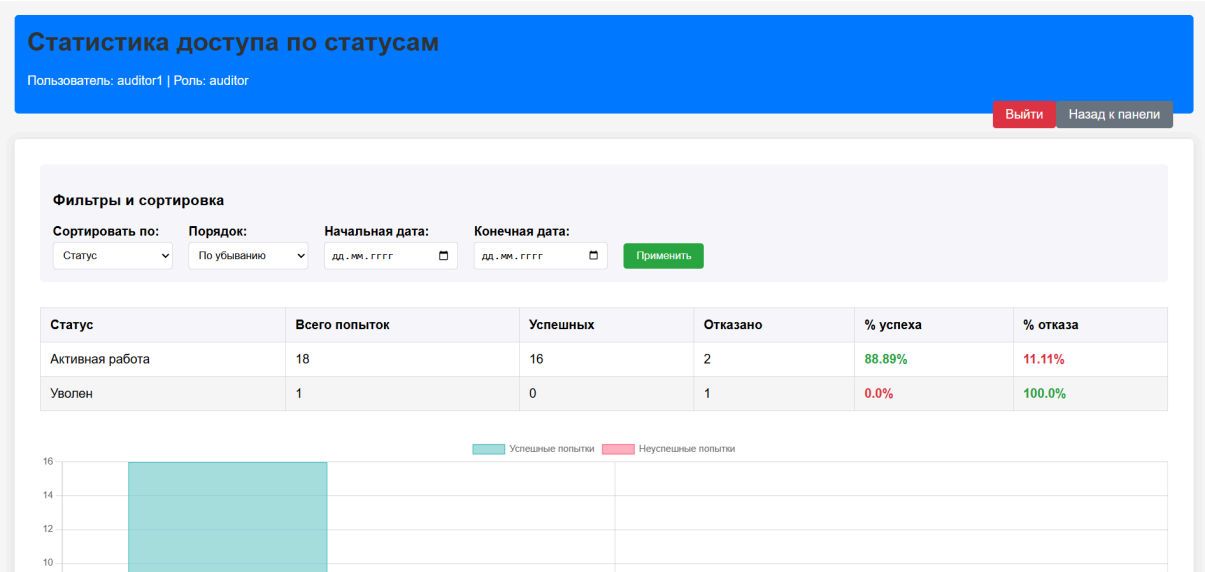


Рисунок 14 – Статистика аудитора Дементьева Сергея Даниловича

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы успешно реализован веб-интерфейс для системы управления доступом с поддержкой многопользовательского режима. Приложение обеспечивает:

- безопасную аутентификацию на основе JWT и ролевую модель доступа;
- персонализированные интерфейсы для 5 категорий пользователей;
- сложную аналитику с использованием GROUP BY, сортировкой и фильтрацией;
- полное соответствие ранее разработанной структуре БД и политикам безопасности;
- масштабируемую архитектуру на основе FastAPI и SQLAlchemy;

Реализация демонстрирует практические навыки работы с современным стеком веб-разработки и интеграции механизмов информационной безопасности на уровне приложения.